7 Retrieval Augmented Generation

How the Three Forms Compare

The result-generation stage is the clearest fork in a RAG design, so it is worth seeing its three forms side by side.

Form	How it answers	Best for	Main risk
Reader	Marks and returns the answer span verbatim from one passage	Exact facts, short passages, high-stakes answers	Cannot combine or rephrase; fails if no single span holds the answer
Generator	Writes a new, direct answer from several passages with a language model	Open-ended questions, fluent answers across sources	Hallucination; must be grounded by prompt and guardrails
Synthesizer	Loops retrieval and reasoning over several documents, then summarizes	Deep research needing evidence gathered across sources	Latency and complexity; drift across hops

The progression is one of flexibility traded against faithfulness. The reader cannot say anything its source did not, which makes it safe but rigid; the synthesizer can assemble an answer no single source contains, which makes it powerful but the hardest to keep honest. The form is chosen per query by the context available, the question's complexity, the user's intent, and how costly a wrong answer would be. In an agentic system that choice moves into the loop itself.

Key Takeaways

1. A ranked list leaves the user to find the answer, and a language model alone cannot answer what it never saw. RAG combines the two: retrieval supplies the evidence, the generator turns it into a direct answer, and grounding ties the answer to sources a user can check.
2. A RAG system reuses the retrieval machinery of earlier chapters, BM25, embeddings, vector search, and cross-encoder reranking, and adds two parts that exist because a model, not a human, now consumes the results: two-level chunking and a result-generation stage.
3. Chunking balances sharp embeddings against enough context for the generator. Two-level chunking keeps both, small chunks for matching and larger parents for generation, and no strategy wins everywhere: the right one is the one that retrieves the answers to our own questions.
4. The result-generation stage comes in three forms, a reader that marks a span, a generator that writes a direct answer, and a synthesizer that summarizes across documents. For the generator, the prompt is a real lever on quality, not boilerplate.
5. Agentic RAG replaces one hand-wired pipeline per intent with a single reason-act-observe loop over declared tools. The loop detects the intent, rewrites the query for each tool, follows multi-hop chains no one coded in advance, and stops within a step budget, with guardrails at its input and output.
6. Evaluating RAG means more than precision and recall. Retrieval still sets the ceiling, but answer quality has four dimensions, correctness, faithfulness, relevance, and completeness, and latency and cost must justify the answer. Two configurations are compared on the same benchmark with a paired test over the questions where they disagree.

Key Formulas

Faithfulness

$$\text{faithfulness} = \frac{|\{\text{claims in the answer supported by the context}\}|}{|\{\text{claims in the answer}\}|} \quad (7.1)$$

see Section 7.5

The share of the answer's claims that the retrieved passages support. It measures grounding, not truth: a correct claim the context does not back still lowers it, and a fully faithful answer can be wrong if the sources are.

Exam focus

- Why a ranked list falls short for single-answer questions, why parametric knowledge fails on recent or private facts, and when RAG is not the right tool.
- Two-level chunking, and why hybrid retrieval over two chunk levels must reconcile both rankings to a common unit before rank fusion.
- Reader versus generator versus synthesizer: mechanism, strengths, and failure modes.
- Fixed pipeline versus agent loop: what the loop takes over (intent, query transformation, multi-hop expansion, stopping) and where input and output guardrails apply.
- Correctness versus faithfulness versus relevance, and why an LLM judge must be validated against human judgements.
- Comparing two configurations: why only the disagreements count, and why the test asks for a result "at least as extreme" rather than the exact split.

Self-Check Questions

1. (Understand) A user asks a plain language model about an event last week and gets a confident but wrong answer. Explain which limit of parametric knowledge this illustrates and how RAG would address it.
2. (Understand) Dense retrieval runs on small chunks and sparse retrieval on their larger parents. Explain why their result lists cannot be fused with reciprocal rank fusion directly, and what has to happen first.
3. (Analyze) A RAG system splits documents into 2000-token chunks and retrieval precision is high, but generated answers are vague and incomplete. Diagnose the likely cause and propose a chunking change.
4. (Analyze) An answer scores 0.9 on answer relevance but 0.4 on faithfulness. What does this combination tell you about the system, and where would you look first?
5. (Apply) On a benchmark of 50 questions, a new configuration B answers 41 correctly and the baseline A answers 38. They disagree on 5 questions, of which B wins 4. Compute the two-sided sign-test probability and decide whether B is convincingly better.
6. (Evaluate) You are asked to build a question-answering feature over a product catalogue whose prices and stock change hourly. Argue whether RAG, fine-tuning, or direct structured-data queries is the better fit, and justify the trade-offs.
7. (Evaluate) A support assistant scoped to one insurance product starts answering questions about competitors' policies and sometimes states facts not found in its retrieved passages. Explain which guardrails are missing and where in the agent loop each one belongs.

Test Your Knowledge

Take the Chapter 7 Quiz ->

Further Reading

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). **Retrieval-augmented generation for knowledge-intensive NLP tasks**. *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2005.11401. The paper that named RAG and introduced the retrieve-then-generate architecture this chapter builds on.
- Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., et al. (2020). **Dense passage retrieval for open-domain question answering**. *EMNLP 2020*. arXiv:2004.04906. DPR, the dual-encoder dense retriever behind the dense-retrieval arm of the pipeline.
- Reimers, N., & Gurevych, I. (2019). **Sentence-BERT: Sentence embeddings using Siamese BERT-networks**. *EMNLP 2019*. arXiv:1908.10084. The sentence-embedding method used to embed chunks and queries, and to measure similarity in semantic chunking.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). **BERT: Pre-training of deep bidirectional transformers for language understanding**. *NAACL-HLT 2019*. arXiv:1810.04805. The encoder behind both the reader and the cross-encoder reranker.
- Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). **SQuAD: 100,000+ questions for machine comprehension of text**. *EMNLP 2016*. Dataset. The benchmark used to train and evaluate the reader of this chapter.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). **Lost in the middle: How language models use long contexts**. *Transactions of the Association for Computational Linguistics*.

arXiv:2307.03172. Shows that models use information at the start and end of a long context far better than in the middle, one reason the order of a prompt matters.

- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). **ReAct: Synergizing reasoning and acting in language models**. *ICLR 2023*. arXiv:2210.03629. The reason-act-observe loop underlying the agentic RAG of this chapter.
- Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). **RAGAS: Automated evaluation of retrieval augmented generation**. arXiv:2309.15217. The evaluation framework behind the faithfulness, answer relevance, and context precision and recall metrics of this chapter.
- deepset. **Haystack documentation**. docs.haystack.deepset.ai. Reference for the components and pipelines used to build the RAG pipeline in code.
- Firecrawl (2026). **OpenClaw vs Hermes Agent: Which one should you actually run?** *Firecrawl blog*. firecrawl.dev. A practical comparison of two personal assistants that run a single agent loop for everything a user asks.

Key formulas

Faithfulness

$$\text{faithfulness} = \frac{|\{\text{claims in the answer supported by the context}\}|}{|\{\text{claims in the answer}\}|} \tag{7.1}$$